Metody Inteligencji Obliczeniowej

Sprawozdanie 2

Wydział Fizyki i Informatyki Stosowanej

Mikhail Shupliakou

15.03.2026

Spis treści

1 Wprowadzenie **3**
- 1.1 Perceptron wielowarstwowy . . . 3
- 1.2 Struktura sieci . . . 3
- 1.3 Funkcje aktywacji i zastosowania . . . 3
- 1.4 Proces uczenia . . . 3

2 Zadanie 1 **4**
- 2.1 *Preprocessing* danych . . . 4
  - 2.1.1 Wizualizacja zestawu danych wejściowych . . . 4
  - 2.1.2 *Outliery* . . . 5
  - 2.1.3 Detekcja i eleminacja *Outlierów* z IQR . . . 5
  - 2.1.4 Wizualizacja zbioru danych po eliminacji *outlierów* . . . 6
  - 2.1.5 Skalowanie danych . . . 6
- 2.2 Klasyfikacja . . . 7
  - 2.2.1 Dwie warstwy: 5 neuronów każda . . . 7
  - 2.2.2 Porównanie pracy różnych modeli . . . 8
- 2.3 Wnioski . . . 8

3 Zadanie 2 **9**
- 3.1 Opis zbioru danych . . . 9
- 3.2 Skalowanie danych . . . 10
- 3.3 *Training* modelu . . . 10
- 3.4 Ocena pracy modelu . . . 10
- 3.5 Wnioski . . . 12
- 3.6 Czas pracy funkcji *fit* w zależności od struktury sieci . . . 12

4 Spis Literatury i odpowiednie linki **14**

Sieci Wielowarstwowe

1 Wprowadzenie

1.1 Perceptron wielowarstwowy

Perceptron wielowarstwowy (ang. Multilayer Perceptron, MLP) to najpopularniejszy typ sztucznych sieci neuronowych typu jednokierunkowego (*feedforward*). W przeciwieństwie do sieci rekurencyjnych, w podstawowej wersji MLP nie występują sprzężenia zwrotne.

1.2 Struktura sieci

Sieć MLP składa się z kaskadowo połączonych warstw:

- **Warstwa wejściowa:** Przyjmuje dane w postaci wektora cech $\mathbf{x} \in R^n$.
- **Warstwy ukryte:** Składają się z neuronów McCullocha-Pittsa. Liczba warstw oraz neuronów jest dobierana eksperymentalnie, aby uniknąć efektu **nadmiernego dopasowania** (*overfitting*).
- **Warstwa wyjściowa:** Generuje ostateczny wynik obliczeń.

1.3 Funkcje aktywacji i zastosowania

Wybór funkcji aktywacji $\sigma(z)$ zależy od charakteru rozwiązywanego problemu:

1. **Regresja:** Stosuje się **liniowe** funkcje aktywacji, co pozwala na przewidywanie wartości ciągłych.
2. **Klasyfikacja:** Stosuje się **nieliniowe** funkcje aktywacji (np. sigmoidę lub ReLU). Umożliwia to klasyfikację zbiorów, które nie są liniowo separowalne.

1.4 Proces uczenia

Uczenie sieci (szczególnie w zadaniach klasyfikacji) odbywa się przy pomocy algorytmu **wstecznej propagacji błędów** (*backpropagation*). Proces ten polega na minimalizacji funkcji kosztu $J(\mathbf{w})$ poprzez obliczenie gradientu względem wag:

$$\Delta w_{ij} = -\eta \frac{\partial J}{\partial w_{ij}} \tag{1}$$

gdzie η oznacza współczynnik uczenia (*learning rate*).

MLP stanowi fundament dla bardziej złożonych architektur, takich jak **splotowe sieci neuronowe (CNN)**, wykorzystywane w rozpoznawaniu obrazów. Dzięki warstwom ukrytym i nieliniowości, MLP jest uniwersalnym aproksymatorem dowolnych funkcji ciągłych.

2 Zadanie 1

Celem zadania jest budowa oraz optymalizacja wielowarstwowej sieci neuronowej do klasyfikacji skuteczności nowego leku na podstawie znormalizowanych stężeń jego składników. Analiza obejmuje ocenę wpływu architektury modelu na jakość predykcji, z zachowaniem stratyfikacji zbiorów danych, a także graficzną wizualizację wyznaczonych granic decyzyjnych.

2.1 *Preprocessing* danych

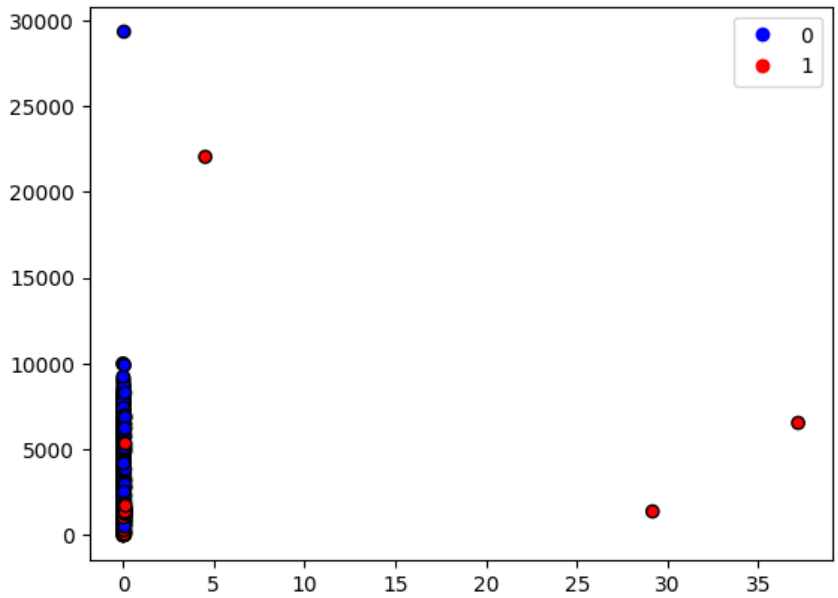
Mając wcześniej przygotowany plik tekstowy z danymi na temat efektywności leczenia, powinienem przeprowadzić analizę danych. Muszę przede wszystkim przeprowadzić pewien *preprocessing* (skalowanie, eliminacja *outlierów*) do dalszej analizy danych.

```
#wprowadzenie danych
df = pd.read_csv('medicine.txt')
print(df.head())
```

Widok danych:

```
   Presence 1  Presence 2  Was medicine effective?
0     0.04609     6010.05                        0
1     0.02237     6119.66                        0
2     0.05054     7037.36                        0
3     0.03155     5009.20                        0
4     0.02256     6214.69                        0
...
```

2.1.1 Wizualizacja zestawu danych wejściowych



Rysunek 1: Zestaw danych na których pracujemy

2.1.2 *Outliery*

Na powyższym wykresie pokazano zestaw danych, nad którymi tak naprawdę pracujemy. Bardzo dobrze widać, że są znaczenia, które bardzo się różnią od innych(Jest ich cztery). Te kropki na wykresie są *outlierami*, które będą miały negatywny wpływ na przeprowadzenie analizy danych. Dlatego dobra praktyką jest pozbawienie się tych znaczeń.

Wartości odstające w analizie danych

Wartości odstające (outliery) to punkty danych znacząco różniące się od reszty zbioru, które nie pasują do ogólnego wzorca. Mogą one powstawać na skutek błędów ludzkich przy wprowadzaniu informacji, wadliwych pomiarów, błędów w przetwarzaniu, a czasem po prostu z powodu naturalnej zmienności badanego zjawiska. Zrozumienie ich źródła jest kluczowe przed podjęciem decyzji o ich ewentualnym usunięciu.

Eliminacja wartości odstających jest często niezbędna dla poprawności analizy. Skrajne obserwacje silnie zniekształcają podstawowe miary statystyczne, takie jak średnia czy wariancja, co może prowadzić do zafałszowanych wniosków i osłabiać wiarygodność całego badania. W przypadku modeli uczenia maszynowego outliery mogą powodować przeuczenie (overfitting) i utrudniać prawidłową interpretację wyników. Odpowiednie oczyszczenie zbioru z anomalii pozwala algorytmom skupić się na głównym trendzie, co bezpośrednio poprawia stabilność, dokładność i zdolność modelu do generalizacji na nowych danych.

2.1.3 Detekcja i eleminacja *Outlierów* z IQR

Do detekcji mogą być użyte różne metody, ale ja wybrałem *Interquartile Range*.

IQR jest stosowany do pomiaru zmienności poprzez podział zbioru danych na kwartyle. Dane są sortowane rosnąco, a następnie dzielone na 4 równe części. Wartości Q_1 (25. percentyl), Q_2 (50. percentyl, czyli mediana) oraz Q_3 (75. percentyl) dzielą zbiór danych na 4 równe obszary. Rozstęp międzykwartylowy oblicza się za pomocą poniższego wzoru:

$$IQR = Q_3 - Q_1$$

Punkty danych, których wartości spadają poniżej $Q_1 - 1.5 \times IQR$ lub przekraczają $Q_3 + 1.5 \times IQR$, klasyfikowane są jako wartości odstające.

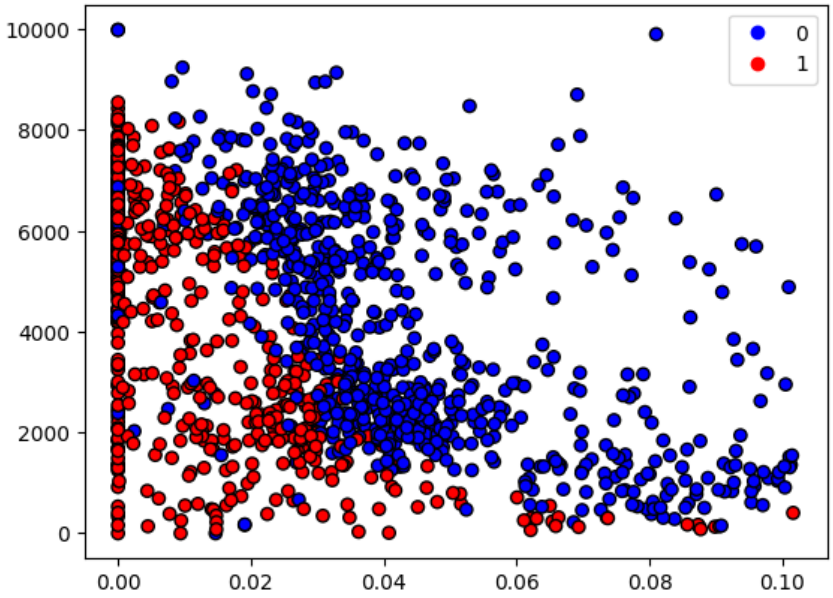
Użyjmy tej metody w kodzie:

```
#OUTLIERS

features = df.columns[:2]

# Liczymy 25 i 75 procentyle
Q1 = df[features].quantile(0.25)
Q3 = df[features].quantile(0.75)
IQR = Q3 - Q1

# Wyliczamy granice
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
# tworzymy filter
condition = ~((df[features] < lower_bound) | (df[features] >
↪  upper_bound)).any(axis=1)

# uzywamy filter
df_clean = df[condition]

print(f"Rozmiar poczatkowy: {len(df)}, Po eliminacji: {len(df_clean)}")
```

```
Rozmiar poczatkowy: 1350, Po eliminacji: 1285
```

2.1.4 Wizualizacja zbioru danych po eliminacji *outlierów*



Rysunek 2: Zbiór danych po eliminacji *outlierów*

2.1.5 Skalowanie danych

W celu ujednolicenia zakresu wartości cech i zapewnienia stabilnego oraz szybkiego procesu uczenia wielowarstwowej sieci neuronowej, zastosowano normalizację metodą skalowania min-max (ang. Min-Max Scaling). Proces ten liniowo przekształca wartości każdej zmiennej w taki sposób, aby mieściły się w standardowym przedziale $[0, 1]$. Wartość znormalizowaną dla każdej obserwacji oblicza się zgodnie z poniższym wzorem:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

gdzie X_{min} oraz X_{max} to odpowiednio najmniejsza i największa wartość danej cechy w analizowanym zbiorze danych.

```
#SCALING PROCESS

from sklearn.preprocessing import MinMaxScaler

X = df.iloc[:, :2]
y = df.iloc[:, 2]
```

```
scaler = MinMaxScaler()
scaled_X = scaler.fit_transform(X)
X = pd.DataFrame(scaled_X, columns=X.columns)

X.head()
```

2.2 Klasyfikacja

2.2.1 Dwie warstwy: 5 neuronów każda

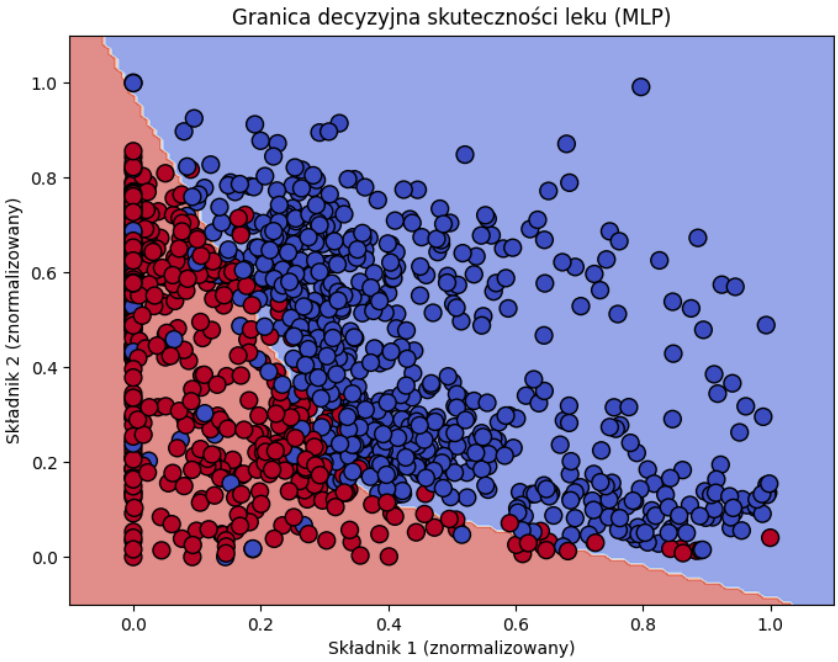
Zbiór danych został podzielony na 80% danych treningowych i 20% testowych. Model MLP został nauczony. Model ma dwie warstwy, każda z których ma 5 neuronów.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↪  random_state=42)

model = MLPClassifier(hidden_layer_sizes = (5,5), max_iter=2000)

# Trenowanie modelu
model.fit(X_train, y_train)

# Ocena modelu
train_accuracy = accuracy_score(y_train, model.predict(X_train))
test_accuracy = accuracy_score(y_test, model.predict(X_test))
```

Ocena modelu:

```
Dokladnosc na zbiorze treningowym: 0.9144
Dokladnosc na zbiorze testowym: 0.9377
```

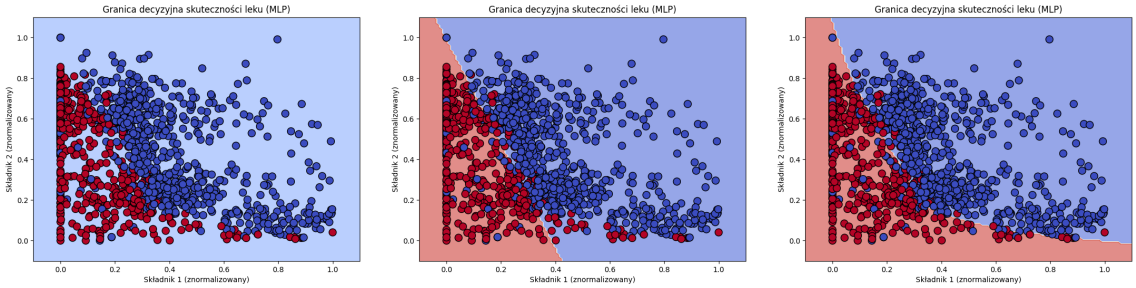
Dokonano wizualizacji podziału sieci danych:



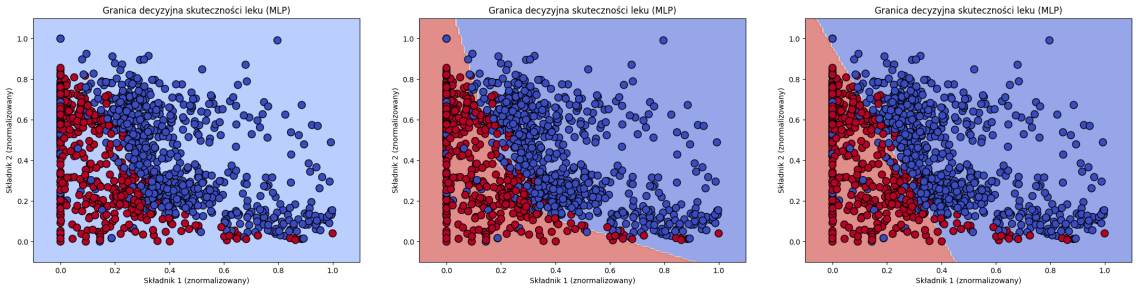
Rysunek 3: Dwie warstwy (5 neuronów każda)

Zadanie polega na porównaniu różnych wyników w zależności od struktury sieci.

2.2.2 Porównanie pracy różnych modeli



(a) (2, 2, 2) (b) (3, 3) (c) (50, 50, 50)



(d) (1, 2) (e) (10) (f) (1, 20)

Rysunek 4: Zbiorcze porównanie wyników dla sześciu różnych architektur.

Tabela 1: Porównanie dokładności (accuracy) dla różnych architektur modelu

Struktura sieci	**Dokładność treningowa**	**Dokładność testowa**
(2, 2, 2)	0.8998	0.9300
(3, 3)	0.9008	0.9261
(50, 50, 50)	0.9086	0.9455
(1, 2)	0.8969	0.9222
(10)	0.9115	0.9300
(1, 20)	0.8998	0.9300

2.3 Wnioski

Na podstawie wizualizacji granic decyzyjnych (Rysunek 4) oraz zestawienia liczbowego (Tabela 1) można wyciągnąć następujące wnioski dotyczące wpływu struktury sieci na jej skuteczność:

- **Wpływ złożoności architektury:** Model o najbardziej rozbudowanej strukturze, składający się z trzech warstw ukrytych po 50 neuronów $(50, 50, 50)$, osiągnął najwyższą dokładność zarówno na zbiorze treningowym (około 0.9086), jak i testowym (0.9455). Analiza wizualna (wykres c) potwierdza, że tak duża pojemność modelu pozwala na wygenerowanie wysoce elastycznej, nieliniowej granicy decyzyjnej, która precyzyjniej dopasowuje się do trudnych obszarów na styku obu klas.

- **Ograniczenia zbyt prostych modeli (zjawisko niedouczenia):** Choć metryki liczbowe dla bardzo prostych architektur mogą wydawać się akceptowalne (np. dokładność 0.9222 na zbiorze testowym dla modelu $(1, 2)$), analiza wizualna ujawnia ich istotne wady. Na wykresie (d) wyraźnie widać, że zastosowanie zaledwie jednego neuronu w pierwszej warstwie ukrytej tworzy tzw. "wąskie gardło"(bottleneck). Drastycznie ogranicza to elastyczność sieci, przez co granica decyzyjna staje się niemal liniowa i nie radzi sobie z poprawnym oddzieleniem nieliniowo ułożonych klas. Jest to klasyczny przykład niedopasowania (underfitting), który udowadnia, że ocena modelu wyłącznie na podstawie samej dokładności (accuracy) bywa złudna.
- **Optymalny kompromis:** Złoty środek pomiędzy złożonością a skutecznością prezentują modele o umiarkowanej pojemności, takie jak sieć z jedną warstwą ukrytą o 10 neuronach (model (10)). Zapewnia on wysoką skuteczność (0.9300), a jego granica decyzyjna (wykres e) gładko i trafnie odwzorowuje rzeczywisty kształt danych, unikając zarówno niedouczenia (jak w modelu $(1, 2)$), jak i nadmiernego skomplikowania.
- **Relacja dokładności treningowej i testowej:** We wszystkich badanych wariantach dokładność testowa przewyższa dokładność treningową. Jest to sytuacja odwrotna do typowego zjawiska przeuczenia (overfitting). Może to sugerować specyficzny podział danych (gdzie zbiór testowy okazał się "łatwiejszy"lub bardziej jednorodny), bądź skuteczne działanie mechanizmów regularyzacji. Mimo rozbudowy sieci (model 50, 50, 50), nie zaobserwowano utraty zdolności do generalizacji.

3 Zadanie 2

Celem niniejszego zadania jest pobranie, samodzielne wstępne przetworzenie oraz analiza silnie niezbalansowanego zbioru danych Yeast z repozytorium UCI. Głównym zadaniem jest wytrenowanie sieci neuronowych o różnych konfiguracjach, zmierzenie czasu ich uczenia oraz osiągnięcie jak najlepszych wyników klasyfikacji. W celu oceny modeli należy wygenerować macierze pomyłek dla zbiorów uczącego i testowego, obliczyć wszystkie poznane dotychczas metryki jakości oraz przeanalizować, czy dokładność (accuracy) na poziomie 0.5 jest akceptowalna dla tego zbioru danych.

3.1 Opis zbioru danych

```python
from ucimlrepo import fetch_ucirepo

# fetch dataset
yeast = fetch_ucirepo(id=110)

# data (as pandas dataframes)
X = yeast.data.features
y = yeast.data.targets
```

Powyższy fragment kodu wykorzystuje bibliotekę `ucimlrepo` do pobrania zbioru danych „Yeast” (identyfikator 110) z repozytorium UCI Machine Learning. Pobrane dane są następnie przypisywane do dwóch zmiennych w formacie `pandas.DataFrame`: macierzy cech (X) oraz wektora zmiennych docelowych (y).

Sam zbiór danych dotyczy problemu klasyfikacji wieloklasowej w bioinformatyce. Jego celem jest przewidywanie miejsca lokalizacji białek w komórkach drożdży (np. cytoplazma, jądro komórkowe, mitochondria) na podstawie ich właściwości biochemicznych i strukturalnych. Macierz cech (X) zawiera liczbowe oceny różnych sygnałów lokalizacyjnych i sekwencji aminokwasowych, natomiast wektor docelowy (y) przechowuje informacje o docelowej organelli, w której to białko ostatecznie funkcjonuje.

3.2 Skalowanie danych

Kolejnym etapem przetwarzania było skalowanie cech. Najpierw zbiór danych podzielono na część uczącą i testową w proporcji 80:20, z wykorzystaniem stratyfikacji w celu zachowania oryginalnych proporcji klas. Następnie zastosowano `RobustScaler`, który jest odporny na wartości odstające. Aby zapobiec zjawisku wycieku danych (ang. *data leakage*), skaler dopasowano (*fit*) wyłącznie na danych uczących, a dopiero potem za jego pomocą przekształcono (*transform*) zarówno zbiór uczący, jak i testowy.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3.3 *Training* modelu

```
network = MLPClassifier(solver='adam',hidden_layer_sizes=(100, 100), max_iter =
↪  2000, tol = 0.001, activation = 'relu')
network.fit(X_train_scaled,y_train)
```

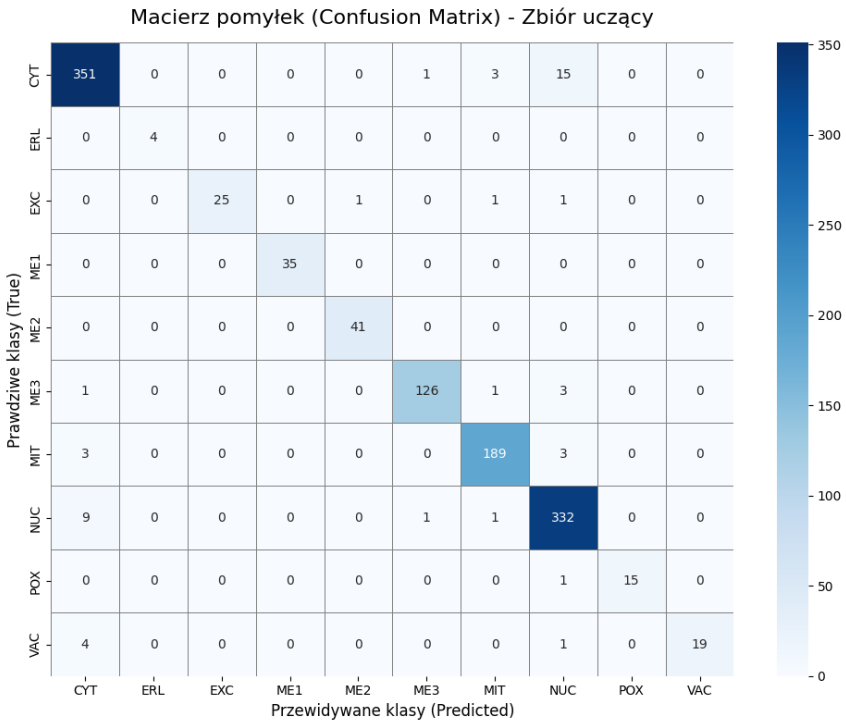
W kolejnym etapie przeprowadzono proces uczenia modelu sieci neuronowej (`MLPClassifier`). Skonfigurowano architekturę z dwoma warstwami ukrytymi po 100 neuronów każda, funkcją aktywacji ReLU oraz optymalizatorem Adam.

3.4 Ocena pracy modelu

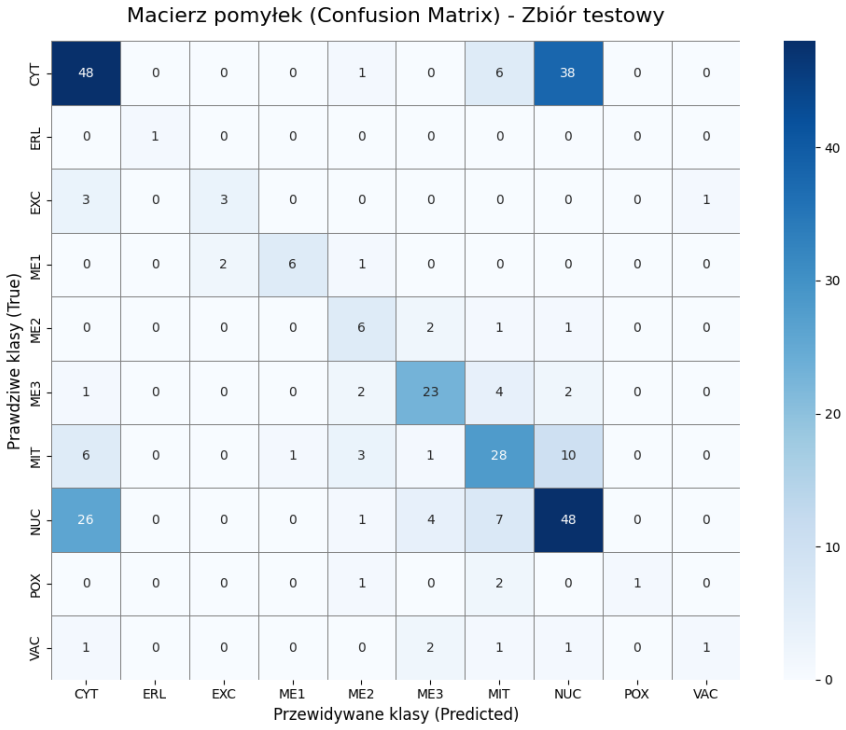
```
=== Wyniki dla zbioru uczcego ===
Dokadno (Accuracy): 0.9579
Precyzja (Precision):  0.9585
Czuo (Recall):      0.9579
Miara F1 (F1-score):   0.9578
```

```
=== Wyniki dla zbioru testowego ===
Dokadno (Accuracy): 0.5556
Precyzja (Precision):  0.5681
Czuo (Recall):      0.5556
Miara F1 (F1-score):   0.5545
```

Otrzymane wyniki ewaluacji wskazują na wyraźne zjawisko przeuczenia (ang. *overfitting*). Model osiągnął bardzo wysoką dokładność na zbiorze uczącym (ok. 95,8%), jednak na zbiorze testowym dokładność spadła do poziomu około 55,6%. Podobne rozbieżności można zaobserwować w pozostałych metrykach: precyzji, czułości oraz mierze F1.



Rysunek 5: Confussion Matrix - Zbiór do nauczenia się

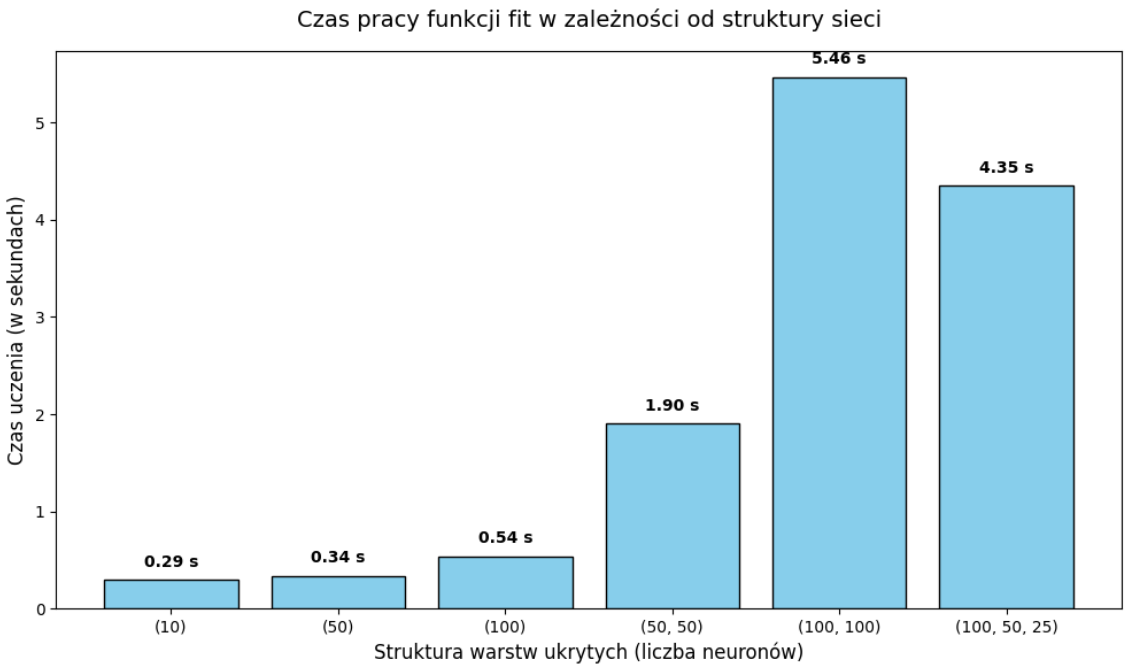


Rysunek 6: Confussion Matrix - Zbiór testowy

3.5 Wnioski

Mimo spadku wydajności na danych testowych, wynik na poziomie ok. 0.55 należy uznać za jak najbardziej satysfakcjonujący i dobry dla zbioru danych Yeast. Wynika to z faktu, że zbiór ten charakteryzuje się dużą nierównowagą klas oraz zawiera aż 10 różnych etykiet. Klasyfikacja w pełni losowa dałaby wynik na poziomie zaledwie 10%, a naiwne przypisywanie zawsze klasy dominującej (CYT) zapewniłoby dokładność rzędu 31%. Osiągnięcie dokładności rzędu 55% stanowi dowód na to, że sieć skutecznie nauczyła się identyfikować nieliniowe zależności biochemiczne w danych. Aby w przyszłości zredukować efekt przeuczenia, można rozważyć zastosowanie silniejszej regularyzacji (np. zwiększenie parametru `alpha`).

3.6 Czas pracy funkcji *fit* w zależności od struktury sieci



Rysunek 7: Czas pracy funkcji *fit* w zależności od struktury sieci

Analiza zaprezentowanego wykresu ukazuje wyraźną zależność między skomplikowaniem architektury sieci neuronowej a czasem trwania procesu jej uczenia.

Sieci posiadające tylko jedną warstwę ukrytą (10, 50 oraz 100 neuronów) uczą się zdecydowanie najszybciej. Czas wykonania funkcji `fit` jest w ich przypadku krótki (poniżej 1 sekundy) i rośnie stosunkowo wolno wraz ze zwiększaniem liczby neuronów.

Dodanie drugiej warstwy ukrytej powoduje gwałtowny skok czasu obliczeń. Przykładowo, zmiana architektury z (50,) na (50, 50) wydłużyła czas uczenia z 0.34 s do 1.90 s. Wynika to z drastycznego wzrostu liczby wag (połączeń między neuronami), które algorytm wstecznej propagacji błędu musi aktualizować w każdej epoce.

Najbardziej wymagająca obliczeniowo okazała się struktura (100, 100), osiągając czas 5.46 s. Niezwykle ciekawym zjawiskiem jest fakt, że głębsza sieć o trzech warstwach (100, 50, 25) uczyła się szybciej (4.35 s) niż sieć (100, 100). Taki rezultat tłumaczy struktura połączeń: sieć (100, 50, 25) posiada sumarycznie mniej

parametrów do optymalizacji niż dwie masywne warstwy po 100 neuronów. Dodatkowo „lejkowa” struktura (zmniejszająca się liczba neuronów w kolejnych warstwach) mogła pomóc optymalizatorowi w szybszym osiągnięciu zbieżności i wcześniejszym zakończeniu procesu uczenia.

4 Spis Literatury i odpowiednie linki

- Perceptron – Wikipedia, wolna encyklopedia
- Perceptron wielowarstwowy - Wikipedia, wolna encyklopedia
- What are Outliers in Data? - geeksforgeeks.org
- Interquartile Range to Detect Outliers in Data -geeksforgeeks.org